



개정4판

모두의 딥러닝

둘째마당

예측 모델의 기본 원리

5장 선형 회귀 모델: 먼저 굿고 수정하기

- 1 경사 하강법의 개요
- 2 파이썬 코딩으로 확인하는 선형 회귀
- 3 다중 선형 회귀의 개요
- 4 파이썬 코딩으로 확인하는 다중 선형 회귀
- 5 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

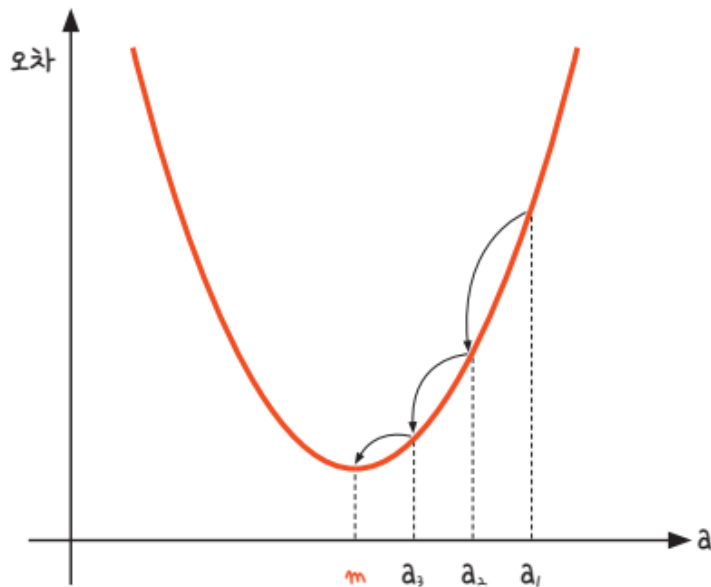
1 경사 하강법의 개요

1 경사 하강법의 개요

● 경사 하강법의 개요

- 기울기 a 를 너무 크게 잡으면 오차가 커지고 기울기를 너무 작게 잡아도 오차가 커짐
- 이때 기울기가 무한대로 커지거나 무한대로 작아지면 그래프는 y 축과 나란한 직선이 됨
- 오차도 함께 무한대로 커짐
- 기울기 a 와 오차 사이에는 그림 5-1의 빨간색 그래프와 같은 이차 함수의 관계가 있다는 의미

▼ 그림 5-1 | 기울기 a 와 오차의 관계: 적절한 기울기를 찾았을 때 오차가 최소화된다

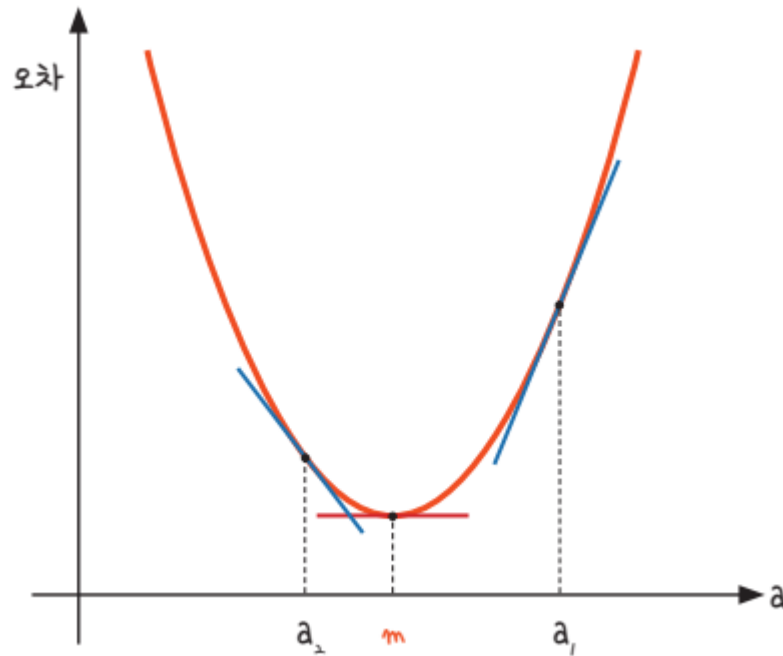


1 경사 하강법의 개요

- 경사 하강법의 개요

- $y = x^2$ 그래프에서 x 에 다음과 같이 a_1, a_2 그리고 m 을 대입해 그 자리에서 미분하면 그림 5-2와 같이 각 점에서의 순간 기울기가 그려짐

▼ 그림 5-2 | 순간 기울기가 0인 점이 곧 우리가 찾는 최솟값 m 이다



1 경사 하강법의 개요

● 경사 하강법의 개요

- 여기서 눈여겨보아야 할 것은 우리가 찾는 최솟값 m 에서의 순간 기울기
- 그래프가 이차 함수 포물선이므로 꼭짓점의 기울기는 x 축과 평행한 선이 되는데 즉, 기울기가 0
- 우리가 할 일은 ‘미분 값이 0인 지점’을 찾는 것이 됨
- 이를 위해 다음 과정을 거침

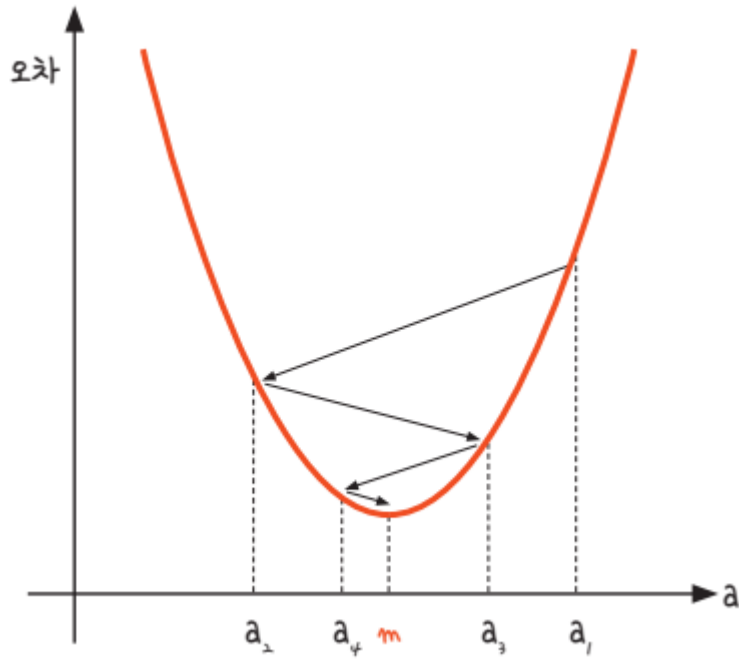
1 | a_1 에서 미분을 구함

2 | 구한 기울기의 반대 방향(기울기가 +면 음의 방향, -면 양의 방향)으로 얼마간 이동시킨 a_2 에서 미분을 구함(그림 5-3 참조)

3 | 앞에서 구한 미분 값이 0이 아니면 1과 2과정을 반복

1 경사 하강법의 개요

▼ 그림 5-3 | 최솟점 m 을 찾아가는 과정

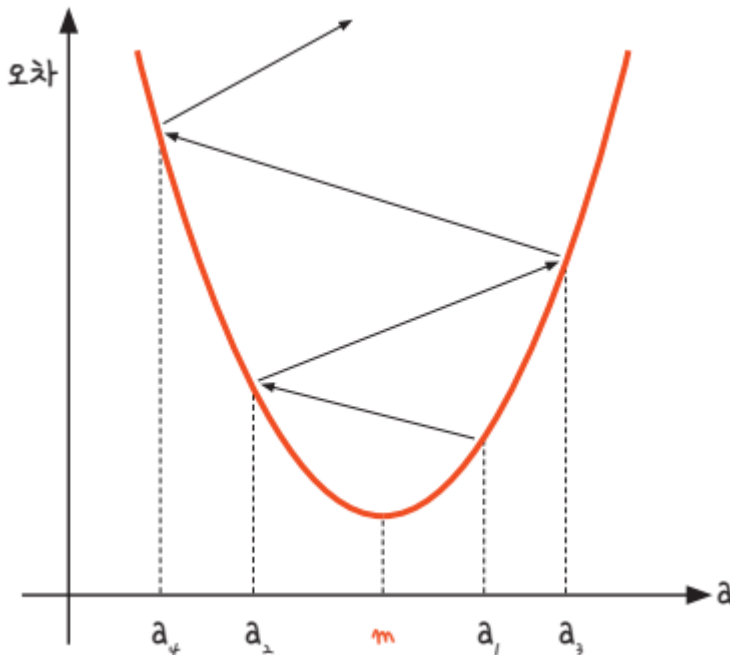


1 경사 하강법의 개요

● 경사 하강법의 개요

- 경사 하강법은 이렇게 반복적으로 기울기 a 를 변화시켜서 m 값을 찾아내는 방법
- 여기서 우리는 학습률(learning rate)이라는 개념을 알 수 있음

▼ 그림 5-4 | 학습률을 너무 크게 잡으면 한 점으로 수렴하지 않고 발산



1 경사 하강법의 개요

● 경사 하강법의 개요

- 어느 만큼 이동시킬지 신중히 결정해야 하는데, 이때 이동 거리를 정해 주는 것이 바로 학습률
- 딥러닝에서 학습률의 값을 적절히 바꾸면서 최적의 학습률을 찾는 것은 중요한 최적화 과정 중 하나
- 다시 말해 경사 하강법은 오차의 변화에 따라 이차 함수 그래프를 만들고 적절한 학습률을 설정해 미분 값이 0인 지점을 구하는 것
- y 절편 b 의 값도 이와 같은 성질을 가지고 있음
- b 값이 너무 크면 오차도 함께 커지고, 너무 작아도 오차가 커짐
- 최적의 b 값을 구할 때 역시 경사 하강법을 사용

2 파이썬 코딩으로 확인하는 선형 회귀

2 파이썬 코딩으로 확인하는 선형 회귀

- 파이썬 코딩으로 확인하는 선형 회귀

- 먼저 평균 제곱 오차의 식을 다시 가져와 보자

$$\frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

- 여기서 $\hat{y}_i$ 는 $y = ax + b$ 의 식에 x_i 를 집어넣었을 때 값이므로 $\hat{y}_i = ax_i + b$ 를 대입하면 다음과 같이 바뀜

$$\frac{1}{n} \sum (y_i - (ax_i + b))^2$$

- 이 값을 미분할 때 우리가 궁금한 것은 a와 b라는 것을 기억해야 함
- 식 전체를 미분하는 것이 아니라 필요한 값을 중심으로 미분해야 하기 때문임

$$a \text{로 편미분한 결과} = \frac{2}{n} \sum -x_i (y_i - (ax_i + b))$$

$$b \text{로 편미분한 결과} = \frac{2}{n} \sum -(y_i - (ax_i + b))$$

2 파이썬 코딩으로 확인하는 선형 회귀

- 파이썬 코딩으로 확인하는 선형 회귀
 - 이를 각각 파이썬 코드로 바꾸면 다음과 같음

```
y_pred = a * x + b          # 예측 값을 구하는 식입니다.  
error = y - y_pred          # 실제 값과 비교한 오차를 error로 놓습니다.  
  
a_diff = (2/n) * sum(-x * (error)) # 오차 함수를 a로 편미분한 값입니다.  
b_diff = (2/n) * sum(-(error))    # 오차 함수를 b로 편미분한 값입니다.
```

- 여기에 학습률을 곱해 기존의 a 값과 b 값을 업데이트

```
lr = 0.03                    # 학습률을 정합니다.  
a = a - lr * a_diff          # 학습률을 곱해 기존의 a 값을 업데이트합니다.  
b = b - lr * b_diff          # 학습률을 곱해 기존의 b 값을 업데이트합니다.
```

2 파이썬 코딩으로 확인하는 선형 회귀

- 파이썬 코딩으로 확인하는 선형 회귀

실습 | 선형 회귀 모델 실습



```
import numpy as np
import matplotlib.pyplot as plt

# 공부 시간 X와 성적 y의 넘파이 배열을 만듭니다.
x = np.array([2, 4, 6, 8])
y = np.array([81, 93, 91, 97])

# 데이터의 분포를 그래프로 나타냅니다.
plt.scatter(x, y)
plt.show()
```

2 파이썬 코딩으로 확인하는 선형 회귀

- 파이썬 코딩으로 확인하는 선형 회귀

```
# 기울기 a의 값과 절편 b의 값을 초기화합니다.
```

```
a = 0
```

```
b = 0
```

```
# 학습률을 정합니다.
```

```
lr = 0.03
```

```
# 몇 번 반복될지 설정합니다.
```

```
epochs = 2001
```

```
# x 값이 총 몇 개인지 셉니다.
```

```
n = len(x)
```

2 파이썬 코딩으로 확인하는 선형 회귀

● 파이썬 코딩으로 확인하는 선형 회귀

```
# 경사 하강법을 시작합니다.
for i in range(epochs):
    y_pred = a * x + b
    error = y - y_pred

    # 에포크 수만큼 반복합니다.
    # 예측 값을 구하는 식입니다.
    # 실제 값과 비교한 오차를 error로 놓습니다.

    a_diff = (2/n) * sum(-x * (error)) # 오차 함수를 a로 편미분한 값입니다.
    b_diff = (2/n) * sum(-(error))      # 오차 함수를 b로 편미분한 값입니다.

    a = a - lr * a_diff # 학습률을 곱해 기존의 a 값을 업데이트합니다.
    b = b - lr * b_diff # 학습률을 곱해 기존의 b 값을 업데이트합니다.

    if i % 100 == 0:      # 100번 반복될 때마다 현재의 a 값, b 값을 출력합니다.
        print("epoch=%.f, 기울기=%.04f, 절편=%.04f" % (i, a, b))
```

2 파이썬 코딩으로 확인하는 선형 회귀

● 파이썬 코딩으로 확인하는 선형 회귀

앞서 구한 최종 a 값을 기울기, b 값을 y 절편에 대입해 그래프를 그립니다.

```
y_pred = a * x + b
```

그래프를 출력합니다.

```
plt.scatter(x, y)
```

```
plt.plot(x, y_pred, 'r')
```

```
plt.show()
```

실행 결과

epoch=0, 기울기=27.8400, 절편=5.4300

epoch=100, 기울기=7.0739, 절편=50.5117

epoch=200, 기울기=4.0960, 절편=68.2822

... (중략) ...

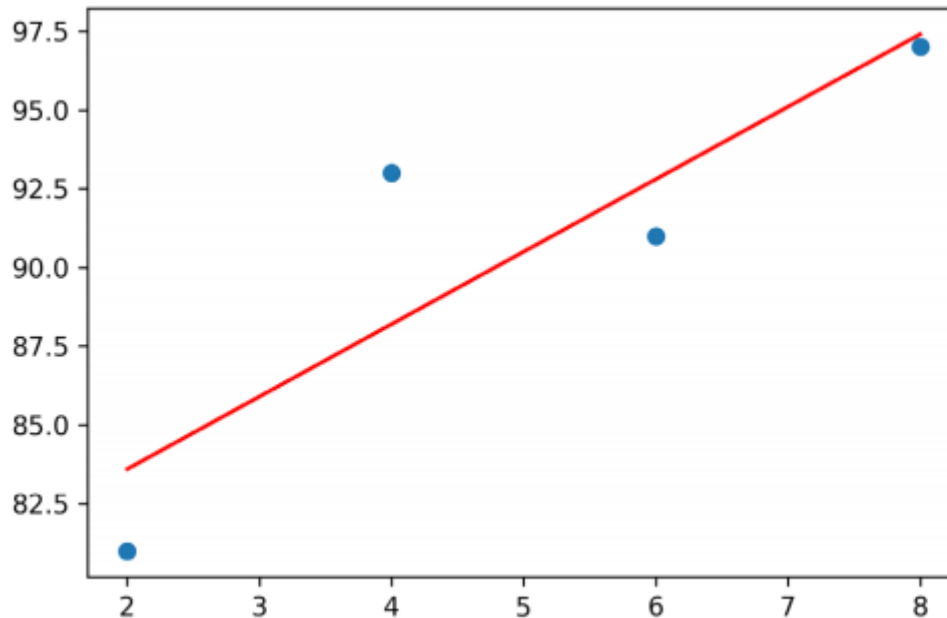
epoch=1900, 기울기=2.3000, 절편=79.0000

epoch=2000, 기울기=2.3000, 절편=79.0000

2 파이썬 코딩으로 확인하는 선형 회귀

- 파이썬 코딩으로 확인하는 선형 회귀

▼ 그림 5-5 | 그래프로 표현한 모습



3 다중 선형 회귀의 개요

3 다중 선형 회귀의 개요

- 다중 선형 회귀의 개요

- 더 정확한 예측을 하려면 추가 정보를 입력해야 하며, 정보를 추가해 새로운 예측 값을 구하려면 변수 개수를 늘려 다중 선형 회귀를 만들어 주어야 함

▼ 표 5-1 | 공부한 시간, 과외 수업 횟수에 따른 성적 데이터

공부한 시간(x_1)	2	4	6	8
과외 수업 횟수(x_2)	0	4	2	3
성적(y)	81	93	91	97

- 독립 변수가 x_1 과 x_2 , 이렇게 두 개 생긴 것
- 이를 사용해 종속 변수 y 를 만들 경우 기울기를 두 개 구해야 하므로 다음과 같은 식이 나옴

$$y = a_1x_1 + a_2x_2 + b$$

4 파이썬 코딩으로 확인하는 다중 선형 회귀

4 파이썬 코딩으로 확인하는 다중 선형 회귀

- 파이썬 코딩으로 확인하는 다중 선형 회귀

- x 값이 두 개이므로 다음과 같이 공부 시간 x_1 , 과외 시간 x_2 , 성적 y 의 넘파이 배열을 만들

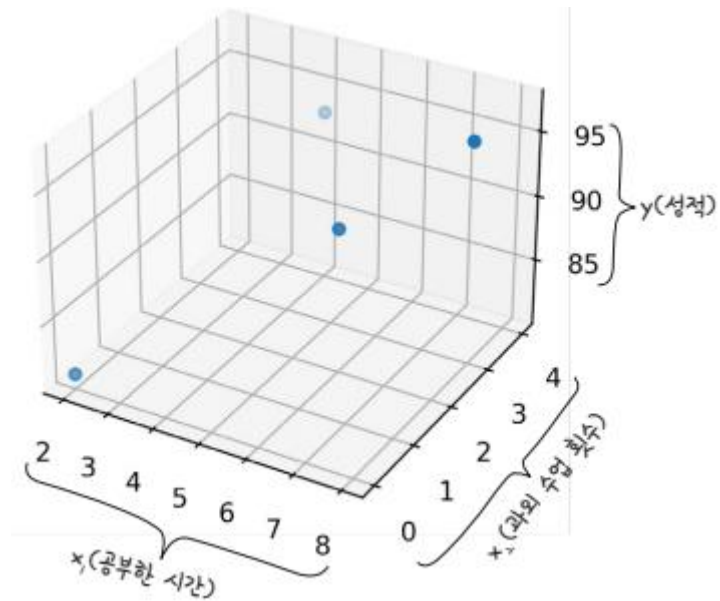
```
x1 = np.array([2, 4, 6, 8])  
x2 = np.array([0, 4, 2, 3])  
y = np.array([81, 93, 91, 97])
```

- 데이터의 분포를 그래프로 표현해 보면 다음과 같음

```
fig = plt.figure()  
ax = fig.add_subplot(111, projection='3d')  
ax.scatter3D(x1, x2, y)  
plt.show()
```

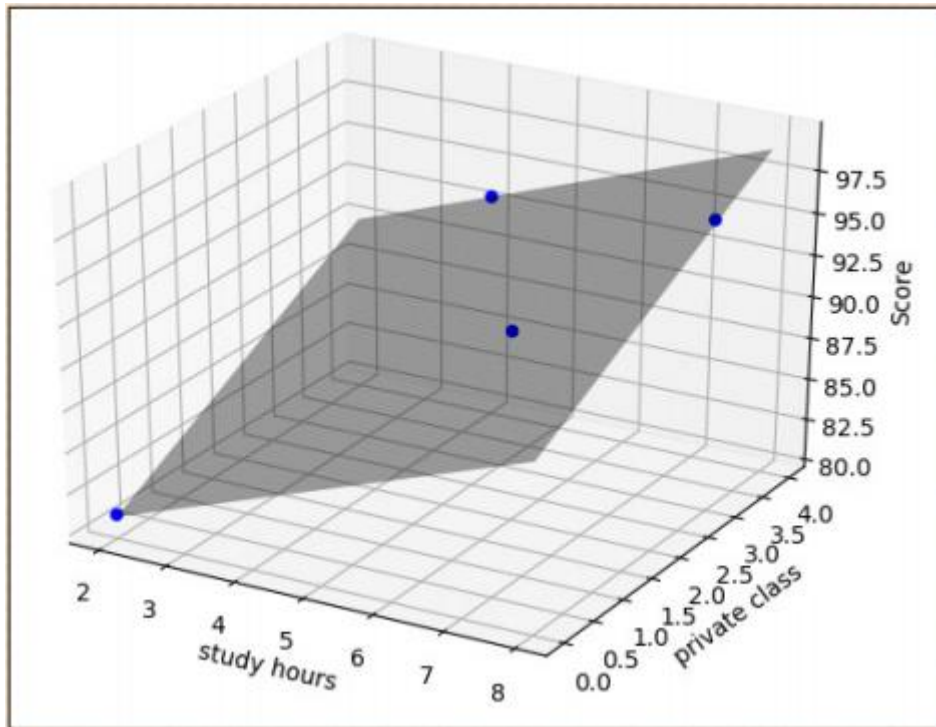
4 파이썬 코딩으로 확인하는 다중 선형 회귀

▼ 그림 5-6 | 축이 하나 더 늘어 3D로 배치된 모습



4 파이썬 코딩으로 확인하는 다중 선형 회귀

▼ 그림 5-7 | 다중 선형 회귀



4 파이썬 코딩으로 확인하는 다중 선형 회귀

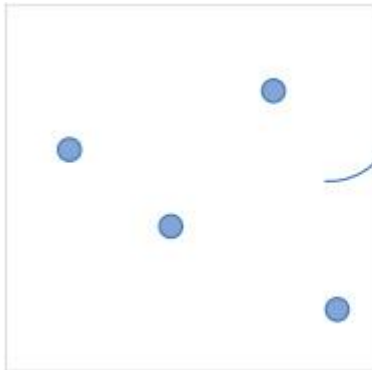
● 파이썬 코딩으로 확인하는 다중 선형 회귀

- 직선상에서 예측하던 것이 평면으로 범위가 넓어지므로 계산이 복잡해지고 더 많은 데이터를 필요로 하게 됨

단순 선형 회귀



다중 선형 회귀



빈 공간이 많아진다.

→ 계산이 복잡하다.

→ 더 많은 데이터가 필요하다.

4 파이썬 코딩으로 확인하는 다중 선형 회귀

● 파이썬 코딩으로 확인하는 다중 선형 회귀

- 코드의 형태는 크게 다르지 않고 다만 x 가 두 개가 되었으므로 x_1, x_2 두 변수를 만들고, 기울기도 a_1 과 a_2 이렇게 두 개를 만듦
- 앞서 했던 방법대로 경사 하강법을 적용해 보자
- 먼저 예측 값을 구하는 식을 다음과 같이 세움

```
y_pred = a1 * x1 + a2 * x2 + b # 기울기와 절편 자리에 a1, a2, b를 각각 대입합니다.
error = y - y_pred             # 실제 값과 비교한 오차를 error로 놓습니다.
```

- 오차 함수를 a_1, a_2, b 로 각각 편미분한 값을 $a1_diff, a2_diff, b_diff$ 라고 할 때 이를 구하는 식은 다음과 같음

```
n = len(x1)                                # 변수의 총 개수입니다.
a1_diff = (2/n) * sum(-x1 * (error))       # 오차 함수를 a1로 편미분한 값입니다.
a2_diff = (2/n) * sum(-x2 * (error))       # 오차 함수를 a2로 편미분한 값입니다.
b_diff = (2/n) * sum(-(error))             # 오차 함수를 b로 편미분한 값입니다.
```

4 파이썬 코딩으로 확인하는 다중 선형 회귀

- 파이썬 코딩으로 확인하는 다중 선형 회귀
 - 학습률을 곱해 기존의 기울기와 절편을 업데이트한 값을 구함

```
a1 = a1 - lr * a1_diff    # 학습률을 곱해 기존의 a1 값을 업데이트합니다.  
a2 = a2 - lr * a2_diff    # 학습률을 곱해 기존의 a2 값을 업데이트합니다.  
b = b - lr * b_diff       # 학습률을 곱해 기존의 b 값을 업데이트합니다.
```

- 이제 실제 점수와 예측된 점수를 출력해서 예측이 잘되는지 확인

```
print("실제 점수: ", y)  
print("예측 점수: ", y_pred)
```

4 파이썬 코딩으로 확인하는 다중 선형 회귀

- 파이썬 코딩으로 확인하는 다중 선형 회귀

실습 | 파이썬 코딩으로 확인하는 다중 선형 회귀



```
import numpy as np
import matplotlib.pyplot as plt

# 공부 시간 x1과 과외 시간 x2, 성적 y의 넘파이 배열을 만듭니다.
x1 = np.array([2, 4, 6, 8])
x2 = np.array([0, 4, 2, 3])
y = np.array([81, 93, 91, 97])
```

4 파이썬 코딩으로 확인하는 다중 선형 회귀

- 파이썬 코딩으로 확인하는 다중 선형 회귀

```
# 데이터의 분포를 그래프로 나타냅니다.  
fig = plt.figure()  
ax = fig.add_subplot(111, projection='3d')  
ax.scatter3D(x1, x2, y);  
plt.show()  
  
# 기울기 a의 값과 절편 b의 값을 초기화합니다.  
a1 = 0  
a2 = 0  
b = 0  
  
# 학습률을 정합니다.  
lr = 0.01
```

4 파이썬 코딩으로 확인하는 다중 선형 회귀

● 파이썬 코딩으로 확인하는 다중 선형 회귀

```
# 몇 번 반복될지 설정합니다.  
epochs = 2001  
  
# x 값이 총 몇 개인지 셉니다. x1과 x2의 수가 같으므로 x1만 세겠습니다.  
n = len(x1)  
  
# 경사 하강법을 시작합니다.  
for i in range(epochs):  
    # 에포크 수만큼 반복합니다.  
    y_pred = a1 * x1 + a2 * x2 + b # 예측 값을 구하는 식을 세웁니다.  
    error = y - y_pred # 실제 값과 비교한 오차를 error로 놓습니다.  
    a1_diff = (2/n) * sum(-x1 * (error)) # 오차 함수를 a1로 편미분한 값입니다.  
    a2_diff = (2/n) * sum(-x2 * (error)) # 오차 함수를 a2로 편미분한 값입니다.  
    b_diff = (2/n) * sum(-(error)) # 오차 함수를 b로 편미분한 값입니다.
```

4 파이썬 코딩으로 확인하는 다중 선형 회귀

● 파이썬 코딩으로 확인하는 다중 선형 회귀

```
a1 = a1 - lr * a1_diff      # 학습률을 곱해 기존의 a1 값을 업데이트합니다.
a2 = a2 - lr * a2_diff      # 학습률을 곱해 기존의 a2 값을 업데이트합니다.
b = b - lr * b_diff         # 학습률을 곱해 기존의 b 값을 업데이트합니다.

if i % 100 == 0:           # 100번 반복될 때마다 현재의 a1, a2, b의 값을 출력합니다.
    print("epoch=%.f, 기울기1=%.04f, 기울기2=%.04f, 절편=%.04f" % (i, a1,
a2, b))

# 실제 점수와 예측된 점수를 출력합니다.
print("실제 점수: ", y)
print("예측 점수: ", y_pred)
```

4 파이썬 코딩으로 확인하는 다중 선형 회귀

- 파이썬 코딩으로 확인하는 다중 선형 회귀

실행 결과

... (전략) ...

epoch=1700, 기울기1=1.5496, 기울기2=2.3028, 절편=77.5168

epoch=1800, 기울기1=1.5361, 기울기2=2.2982, 절편=77.6095

epoch=1900, 기울기1=1.5263, 기울기2=2.2948, 절편=77.6769

epoch=2000, 기울기1=1.5191, 기울기2=2.2923, 절편=77.7260

실제 점수: [81 93 91 97]

예측 점수: [80.76387645 92.97153922 91.42520875 96.7558749]

5 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

5 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

● 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

- 선형 회귀는 현상을 분석하는 방법의 하나
- 머신 러닝은 이러한 분석 방법을 이용해 예측 모델을 만드는 것
- 예를 들어 함수 $y = ax + b$ 는 공부한 시간과 성적의 관계를 유추하기 위해 필요했던 식
- 이렇게 문제를 해결하기 위해 가정하는 식을 머신 러닝에서는 가설 함수(hypothesis)라고 하며 $H(x)$ 라고 표기
- 기울기 a 는 변수 x 에 어느 정도의 가중치를 곱하는지 결정하므로, 가중치(weight)라고 하며, w 로 표시
- 절편 b 는 데이터의 특성에 따라 따로 부여되는 값이므로 편향(bias)이라고 하며, b 로 표시
- 우리가 앞서 배운 $y = ax + b$ 는 머신 러닝에서 다음과 같이 표기

$$y = ax + b \rightarrow H(x) = wx + b$$

5 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

● 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

- 평균 제곱 오차처럼 실제 값과 예측 값 사이의 오차에 대한 식을 손실 함수(lossfunction)

평균 제곱 오차 → 손실 함수(loss function)

- 최적의 기울기와 절편을 찾기 위해 앞서 경사 하강법을 배웠음
- 딥러닝에서는 이를 옵티마이저(optimizer)라고 함
- 우리가 사용했던 경사 하강법은 딥러닝에서 사용하는 여러 옵티마이저 중 하나였음

경사 하강법 → 옵티마이저(optimizer)

- 이제부터는 손실 함수, 옵티마이저라는 용어를 사용해 설명하겠음
- 먼저 텐서플로에 포함된 케라스 API 중 필요한 함수들을 다음과 같이 불러옴

```
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense
```

5 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

● 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

- Sequential() 함수와 Dense() 함수는 2장에서 이미 소개한 바 있음
- 이 함수를 불러와 선형 회귀를 실행하는 코드는 다음과 같음

```
model.add(Dense(1, input_dim=1, activation='linear')) ---- ①  
model.compile(optimizer='sgd', loss='mse') ---- ②  
model.fit(x, y, epochs=500) ---- ③
```

- ① 먼저 가설 함수는 $H(x) = wx + b$
- 이때 출력되는 값(=성적)이 하나씩이므로 Dense() 함수의 첫 번째 인자에 1이라고 설정
- 입력될 변수(=학습 시간)도 하나뿐이므로 input_dim 역시 1이라고 설정
- 입력된 값을 다음 층으로 넘길 때 각 값을 어떻게 처리할지를 결정하는 함수를 활성화 함수
- activation은 활성화 함수를 정하는 옵션
- 여기에서는 선형 회귀를 다루고 있으므로 'linear'라고 적어 주면 됨

5 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

● 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

- ② 앞서 배운 경사 하강법을 실행하려면 옵티마이저에 `sgd`라고 설정
- 손실 함수는 평균 제곱 오차를 사용할 것이므로 `mse`라고 설정
- ③ 끝으로 앞서 따로 적어 주었던 `epochs` 숫자를 `model.fit()` 함수에 적음
- 학습 시간(`x`)이 입력되었을 때의 예측 점수는 `model.predict(x)`로 알 수 있음
- 예측 점수로 그래프를 그려 보면 다음과 같음

```
plt.scatter(x, y)
plt.plot(x, model.predict(x), 'r')    # 예측 결과를 그래프로 나타냅니다.
plt.show()
```

5 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

- 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

실습 I 텐서플로에서 실행하는 선형 회귀



```
import numpy as np
import matplotlib.pyplot as plt

# 텐서플로의 케라스 API에서 필요한 함수들을 불러옵니다.
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

x = np.array([2, 4, 6, 8])
y = np.array([81, 93, 91, 97])

model = Sequential()
```

5 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

● 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

출력 값, 입력 변수, 분석 방법에 맞게끔 모델을 설정합니다.

```
model.add(Dense(1, input_dim=1, activation='linear'))
```

오차 수정을 위해 경사 하강법(sgd)을, 오차의 정도를 판단하기 위해

평균 제곱 오차(mse)를 사용합니다.

```
model.compile(optimizer='sgd', loss='mse')
```

오차를 최소화하는 과정을 500번 반복합니다.

```
model.fit(x, y, epochs=500)
```

```
plt.scatter(x, y)
```

```
plt.plot(x, model.predict(x), 'r') # 예측 결과를 그래프로 나타냅니다.
```

```
plt.show()
```

5 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

- 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

```
# 임의의 시간을 집어넣어 점수를 예측하는 모델을 테스트해 보겠습니다.  
hour = 7  
input_data = tf.constant([[hour]])  
prediction = model.predict(input_data)[0][0]  
print("%.f시간을 공부할 경우의 예상 점수는 %.02f점입니다." % (hour, prediction))
```

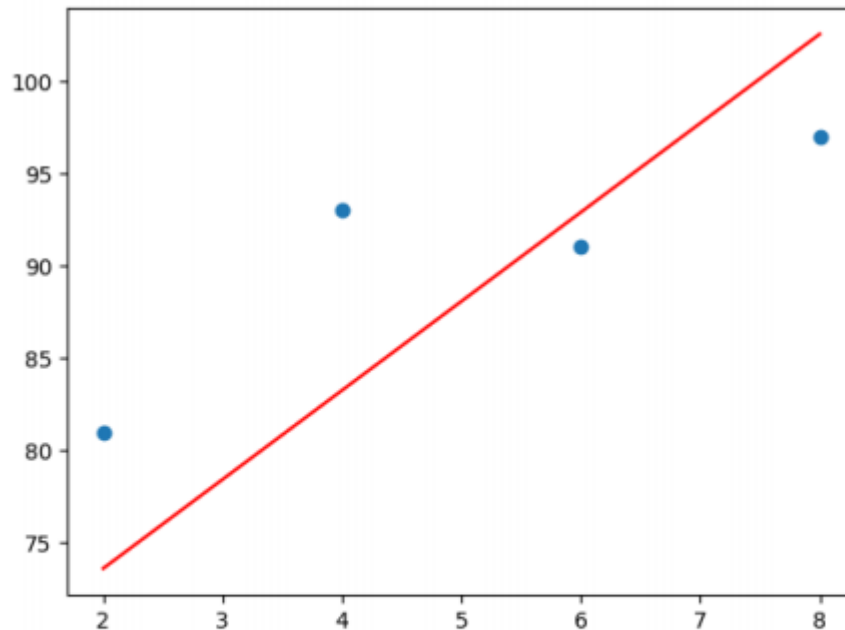
실행 결과

```
Epoch 1/500  
1/1 [=====] - 1s 951ms/step - loss: 8958.2285  
... (중략) ...  
Epoch 500/500  
1/1 [=====] - 0s 2ms/step - loss:46.2887
```

5 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

- 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

▼ 그림 5-8 | 텐서플로로 실행한 선형 회귀 분석 결과



7시간을 공부할 경우의 예상 점수는 97.70점입니다.

5 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

● 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

- 마찬가지로 다중 선형 회귀 역시 텐서플로를 이용해서 실행해 보자
- 다만, 입력해야 하는 변수가 한 개에서 두 개로 늘었음

```
model.add(Dense(1, input_dim=2, activation='linear'))
```

- 변수가 두 개이므로, 모델의 테스트를 위해서도 변수를 두 개 입력
- 임의의 학습 시간과 과외 시간을 입력했을 때의 점수는 다음과 같이 설정해서 구함

```
hour = 6
private_class = 3
input_data = tf.constant([[hour, private_class]])
prediction = model.predict(input_data)[0][0]

print("%.f시간을 공부하고 %.f시간의 과외를 받을 경우, 예상 점수는 %.02f점입니다."
      % (hour, private_class, prediction))
```

5 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

- 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

실습1 텐서플로에서 실행하는 다중 선형 회귀



```
import numpy as np
import matplotlib.pyplot as plt

# 텐서플로의 케라스 API에서 필요한 함수들을 불러옵니다.
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

x = np.array([[2, 0], [4, 4], [6, 2], [8, 3]])
y = np.array([81, 93, 91, 97])

model = Sequential()
```

5 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

- 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

```
# 입력 변수가 두 개(학습 시간, 과외 시간)이므로 input_dim에 2를 입력합니다.
model.add(Dense(1, input_dim=2, activation='linear'))
model.compile(optimizer='sgd', loss='mse')

model.fit(x, y, epochs=500)

# 임의의 학습 시간과 과외 시간을 집어넣어 점수를 예측하는 모델을 테스트해 보겠습니다.
hour = 6
private_class = 3
input_data = tf.constant([[hour, private_class]])
prediction = model.predict(input_data)[0][0]

print("%.f시간을 공부하고 %.f시간의 과외를 받을 경우, 예상 점수는 %.02f점입니다."
      % (hour, private_class, prediction))
```

5 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

- 텐서플로에서 실행하는 선형 회귀, 다중 선형 회귀 모델

실행 결과

Epoch 1/500

1/1 [=====] - 0s 109ms/step - loss: 9019.1279

... (중략) ...

Epoch 500/500

1/1 [=====] - 0s 1ms/step - loss: 39.0065

6시간을 공부하고 3시간의 과외를 받을 경우, 예상 점수는 94.10점입니다.